

GIT & GITHUB

Introduction to Version Control Systems (VCS)

In software development, managing and organizing code changes is essential. Projects can involve multiple developers who work on different parts of the code simultaneously. Without a way to track changes, things can quickly get confusing, and mistakes can easily happen. This is where Version Control Systems (VCS) come in.

Key Points:

- VCS tools help in **tracking, managing, and organizing code changes**.
- They **store different versions** of the code, so you can go back to previous versions if something goes wrong.
- VCS enables **multiple developers** to work on the same project, allowing easy **collaboration**.
- Example: Think of VCS like a "track changes" feature in a Word document, but for code, where you can see who made changes, when, and why.

What is a Version Control System (VCS)?

A Version Control System (VCS) is a tool that helps keep track of the changes made to files over time. It stores every version of the code, so you can revert to earlier versions if needed. It allows developers to work together without overwriting each other's changes.

Benefits of VCS:

1. **Keeps a history of changes:** Every modification is recorded, and you can see who made which change.

2. **Collaboration:** Multiple developers can work on the same project simultaneously without conflicts.
3. **Easy recovery:** If a mistake is made, you can go back to a previous version without losing any work.

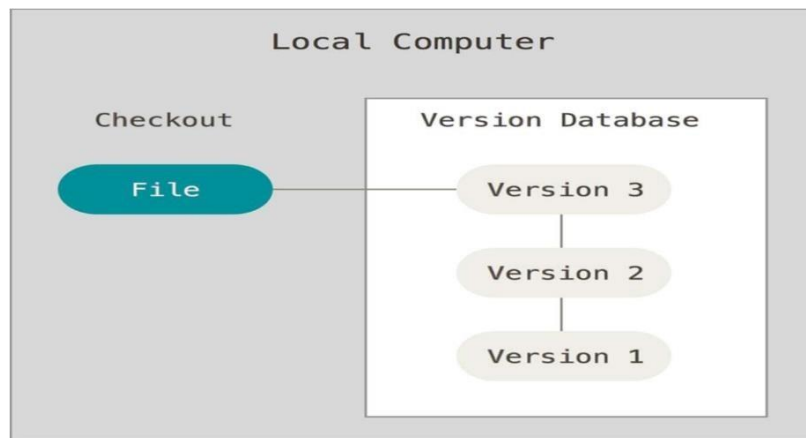
Example: Imagine you're writing a report, and every time you save it, a backup copy is created. You can always go back to an earlier version if the new changes don't work out.

Types of Version Control Systems

There are three main types of VCS, each suited to different situations:

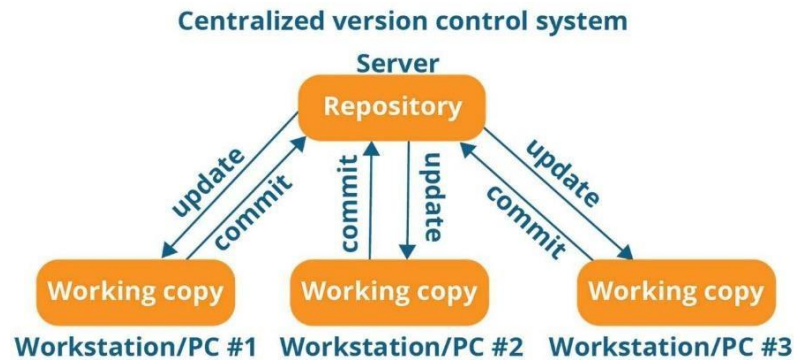
1. **Local Version Control System:**

- Stores versions on your local computer.
- **Problem:** If your computer crashes, you lose everything.
- **Example:** Saving different versions of a file on your desktop.



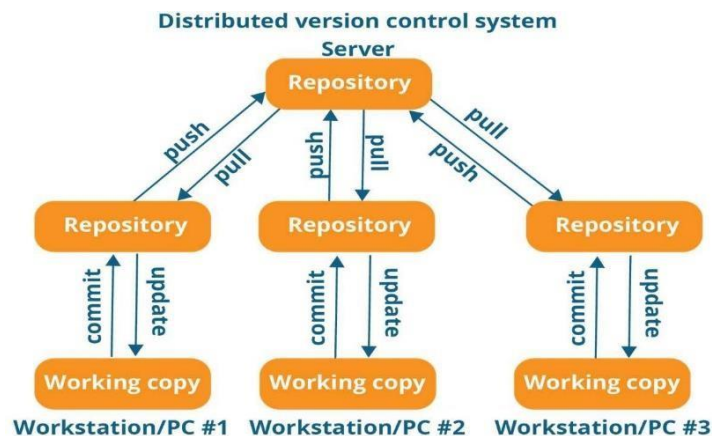
2. **Centralized Version Control System:**

- All versions are stored on a central server. Users connect to this server to access the files.
- **Problem:** If the server crashes, you could lose all your work.
- **Example:** Like a shared folder on Google Drive that everyone can edit.



3. Distributed Version Control System (DVCS):

- Each user has a complete copy of the project, including its entire history.
- **Advantage:** Even if the main server fails, copies exist on multiple devices, ensuring data safety.
- **Example:** Think of it like everyone on the team having their own complete backup of a shared project folder.



Why Distributed VCS is Better:

- Provides **more safety** because data is stored in multiple places.
- Allows **better collaboration** since everyone has a complete copy, making it easy to work offline.

Introduction to Git

As projects grow larger and involve more people, managing changes can become

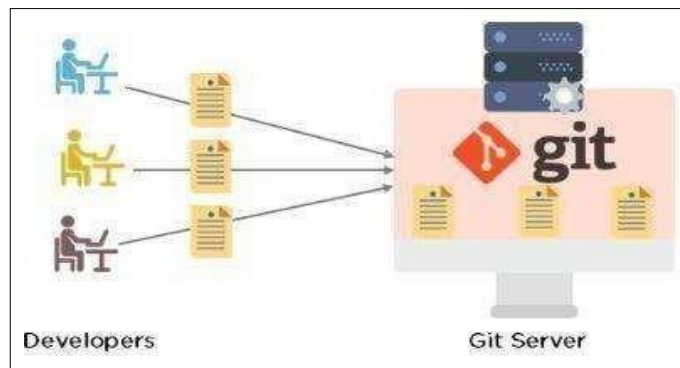
difficult. This is where **Git**, a Distributed VCS, comes in.

What is Git?

- **Git** is a tool that tracks changes in files, especially code, over time.
- It was created by **Linus Torvalds** in **2005** to manage the development of the Linux kernel.
- Git helps multiple people work on the same project simultaneously without causing conflicts.

Features of Git:

1. **Free and open-source:** Anyone can use Git without cost.
2. **Supports collaboration:** Multiple developers can work on the same project at once.
3. **Non-linear development:** Git allows you to create and manage multiple branches of a project.
4. **Tracks entire history:** Every change is saved, so you can go back to any version.



Why Git is Popular:

- Efficiently handles large projects with many contributors.
- Allows for **branching**, where developers can create separate copies of the code to work on features without affecting the main codebase.

How Git Works (Git Workflow)

To use Git effectively, it's important to understand its workflow, which involves three main stages:

1. **Working Directory:**

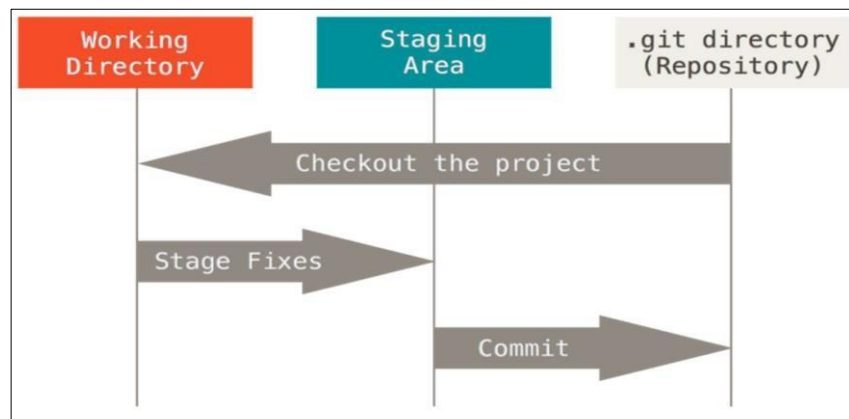
- This is where you make changes to your files.
- **Example:** Editing a file on your computer.

2. **Staging Area:**

- This is where you prepare selected changes to be saved permanently.
- **Example:** Think of it as placing files in a "to-do list" before adding them to your final project.

3. **Git Repository:**

- This is where all your changes are stored, including the entire history of your project.



- **Example:** A folder containing all your backups and old versions of the files.

Steps in Git Workflow:

1. **Edit files** in the Working Directory.
2. **Stage** the files you want to commit (save permanently) in the Staging Area.
3. **Commit** the changes to the Git Repository, where they are saved permanently.

Transition to GitHub: Why Do We Need It?

While Git is great for tracking changes, there's a need for **sharing and collaborating**

on code

across different locations. This is where **GitHub** comes in.

Problem with Just Git:

- If you're working on a project with people who are not in the same place, it's difficult to share changes and keep everything up-to-date.
- GitHub solves this by providing a **platform** where you can **store your Git projects online**, making it easier for teams to collaborate.

What is GitHub?

GitHub is a **web-based platform** that hosts Git repositories, making it easier for teams to **share, collaborate, and manage** their code projects.

Key Points:

1. **Definition:** GitHub is an online service where developers can store, manage, and share their Git projects.
2. **Purpose:** Allows teams to work on the same project from different locations, making collaboration easy.
3. **Owned by:** Microsoft since 2018.
4. **Example:** Imagine GitHub as a social network for programmers where they can share and discuss their code.

Features of GitHub

GitHub offers many features that make project management and collaboration more efficient:

1. **Project Management:** Helps teams track tasks, assign work, and organize projects.
 - **Example:** Like a to-do list that everyone can see and update.
2. **Secure Code Packages:** Safely store and share pieces of code with your team or publicly.
 - **Example:** Uploading a useful function that others can use.

3. **Team Collaboration:** Tools to communicate, review, and manage code changes.
 - **Example:** Commenting on a teammate's code to suggest improvements.
4. **Pull Requests (Code Review):** Discuss and review code changes before merging them into the main project.
 - **Example:** Like submitting a draft for approval before it's published.
5. **Security Tools:** Detect and fix weaknesses in your code.
 - **Example:** Checking for errors before releasing a product.
6. **Code Hosting:** Millions of repositories hosted, with easy access to project details and history.

Alternatives to GitHub

While GitHub is the most popular platform for hosting Git projects, there are other services that provide similar features:

1. **GitLab:** A complete platform for DevOps and collaboration, offering tools for project management and CI/CD.
2. **Bitbucket:** Often used by smaller teams; integrates well with Atlassian tools like Jira.
3. **AWS Code Commit:** Managed Git hosting provided by Amazon Web Services.
4. **SourceForge:** Known for hosting open-source software projects.

Differences Between Git and GitHub

While Git and GitHub are used together, they serve different purposes. Here's how they compare:

Aspect	Git	GitHub
Type	Software tool for tracking code changes	Online platform for hosting Git projects
Interface	Works through the command line	User-friendly graphical web interface
Location	Installed on your local computer	Accessible through a web browser

Purpose	Manages versions and history of code	Helps in sharing, collaborating, and managing projects
Ownership	Developed by Linus Torvalds	Owned by Microsoft
Focus	Version control and tracking code changes	Collaboration, project management, and code sharing
Example	A tool to save and track changes in your project files	A website where you can share your Git projects with others

Git: An essential tool that tracks changes, manages versions, and helps developers work on code without conflicts.

GitHub: A platform that allows teams to share and collaborate on Git projects from anywhere in the world.

BASIC GIT COMMANDS - VERSION, CONFIG, INIT, STATUS, ADD, COMMIT, DIFF, HELP

Basic Git Commands

1. Git Version:

The **git --version** command is used to check if Git is installed on your system.

This command displays the current version of Git that is available.

For example, running: **git --version**

might return `git version 2.37.1`, confirming that Git is installed.

A screenshot of a terminal window titled 'MINGW64: c:/Users/kmit'. The prompt is 'kmit@DESKTOP-50B7B83 MINGW64 ~'. The command '\$ git --version' has been entered, and the output is 'git version 2.37.1.windows.1'.

Knowing the installed Git version helps in troubleshooting and ensuring compatibility with certain Git features or integrations. It is the first step before using Git functionalities.

2. Git Config:

The **git config** command is used to set up the user identity for commits, including name and email address. This ensures that every commit you make in the project history is associated with your correct identity.

Usage Example:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

Explanation:

- **--global:** Applies the configuration globally for the system, meaning it applies to all projects.
- **user.name** and **user.email:** These options specify the identity that will appear in the commit history.

```
kmit@DESKTOP-50B7B83 MINGW64 ~  
$ git config --global user.name "savram674"  
  
kmit@DESKTOP-50B7B83 MINGW64 ~  
$ git config --global user.email "savitharamesh674@gmail.com"  
  
kmit@DESKTOP-50B7B83 MINGW64 ~
```

Example Output:

```
git config --global user.name "Madhurika"  
git config --global user.email "madhurika.kmit@example.com"
```

In this example, Jane Doe's details will be associated with all commits made on this system unless overridden for a specific repository.

3. Git Init:

The **git init** command is used to create a new Git repository. This initializes a directory as a new Git repository and creates a **.git** subdirectory to store all Git-related files.

Usage Example: git init

Explanation: This command creates an empty Git project within the directory. After running **git init**, the project is now under version control, and you can start tracking changes.

```
kmit@DESKTOP-50B7B83 MINGW64 ~/test
$ git init
Initialized empty Git repository in C:/Users/kmit/test/.git/
```

Real-World Example: Imagine you have a directory called **my-project**. Running **git init** inside it will turn it into a Git repository, allowing you to track its changes.

4. Git Status:

The **git status** command displays the state of your working directory and staging area. It shows what changes have been staged, what files are not being tracked by Git, and what changes are yet to be committed.

Usage Example: **git status**

```
MINGW64/c/Users/kmit/test
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        info.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Explanation: This command is essential to get a summary of your repository's current state. It lets you know whether files need to be added to the staging area or if there are changes to commit.

5. Git Add:

The **git add** command is used to add changes to the staging area. You must stage changes before committing them to ensure only the desired changes are included in the commit.

Usage Example: **git add .**

Explanation:

- . stages all changes in the current directory.
- Specific files can be added by replacing . with the file names.

```
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git add .
```

Example: `git add index.html`

This adds the file `index.html` to the staging area.

6. Git Commit:

The `git commit` command saves your changes to the local repository. Each commit represents a snapshot of the repository at a particular point in time. You must include a message explaining what the commit does.

Usage Example: `git commit -m "Added new feature"`

Explanation:

- `-m`: Allows you to add a commit message directly from the command line.
- The commit message should be concise but descriptive enough to explain the change.

```
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git commit -m "info file added"
[master (root-commit) cf73846] info file added
1 file changed, 3 insertions(+)
create mode 100644 info.txt
```

Example Output:

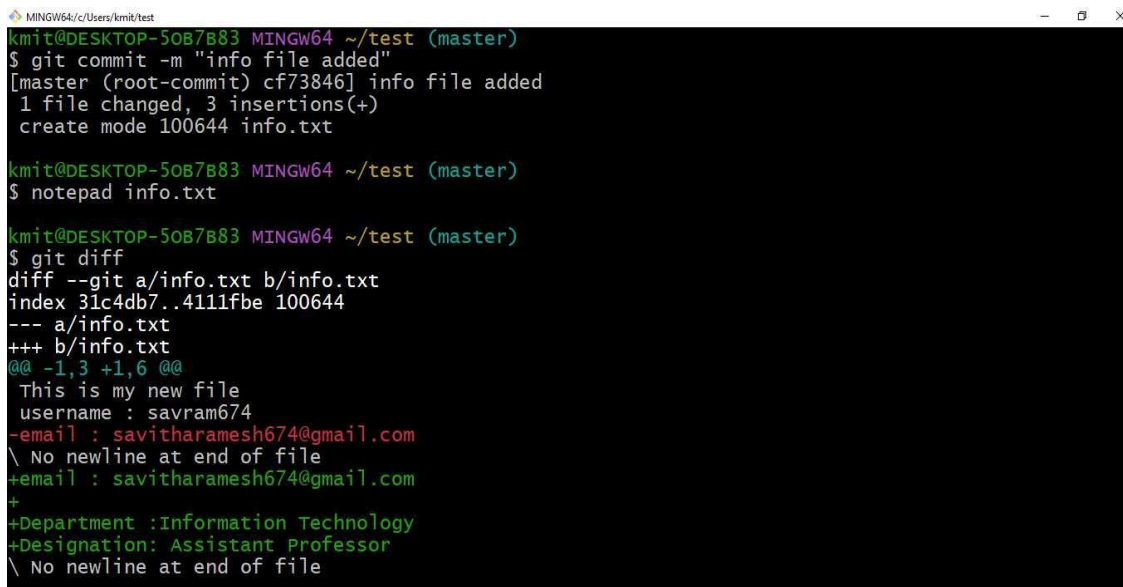
`[master (root-commit) 1a2b3c4] Added new feature`

7. Git Diff:

The **git diff** command shows the differences between files in your working directory and the staging area. It is useful to review changes before committing them.

Usage Example:

git diff

A screenshot of a terminal window titled 'MINGW64/c/Users/kmit/test'. The terminal shows the following commands and output:
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
\$ git commit -m "info file added"
[master (root-commit) cf73846] info file added
1 file changed, 3 insertions(+)
create mode 100644 info.txt

kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
\$ notepad info.txt

kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
\$ git diff
diff --git a/info.txt b/info.txt
index 31c4db7..4111fbc 100644
--- a/info.txt
+++ b/info.txt
@@ -1,3 +1,6 @@
This is my new file
username : savram674
-email : savitharamesh674@gmail.com
\ No newline at end of file
+email : savitharamesh674@gmail.com
+
+Department :Information Technology
+Designation: Assistant Professor
\ No newline at end of file

Explanation: Running **git diff** will display the differences between your modified files and the last commit, helping you see exactly what changes will be staged and committed.

8. Git Help:

The **git help** command provides documentation and detailed usage information for any Git command.

Usage Example: **git help <command>**

Explanation: You can use **git help** to get detailed information about any Git command, such as **git help add** to learn more about the **git add** command.



```
SSPL-LP-DNS-YT0+SimpliLearn@SSPL-LP-DNS-YT01 MINGW64 ~  
$ git help add
```

GIT COMMANDS: WORKING WITH LOCAL AND REMOTE REPOSITORIES - BRANCHES, CHECKOUT, MERGE, REVERT, LOG

Branches in Git

A **branch** in Git is like a **separate copy of your project** where you can make changes without affecting the main version. You might need different branches to work on new features, fix bugs, or prepare for a release.

- Imagine you have a main branch (called **master**) that has the latest stable version of your project. If you need to create a new feature (like adding a search function), you would create a **feature branch**. This allows you to work on the search function independently. Once the feature is complete, you can combine it (merge) with the master branch.

Types of Branches:

- **Main Branch (Master):** The main project everyone uses.
- **Feature Branch:** A branch to work on specific new features.
- **Bug Fix Branch:** Specifically for fixing bugs.
- **Release Branch:** Prepares and tests the project before it is released to users.

Important Commands:

- `git branch <branch_name>`: Creates a new branch, such as `git branch`

search-feature.

- `git branch -a`: Shows a list of all branches, including the main branch, feature branches, and remote branches.
- `git branch -d <branch_name>`: Deletes a branch that you no longer need, such as `git branch -d bugfix`.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
sersact/Sample_Example (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch -a
* master
  project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch -d project_changes
Deleted branch project_changes (was 4e87650).

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch -a
* master
```

2. Switching Between Branches: Checkout

The **git checkout** command is used to **switch between branches**. This allows you to move from one task to another easily. For example, you may be working on a new feature in one branch but need to fix a bug in another branch.

- Let's say you're working on a new search feature in a branch called `search-feature`, but suddenly a critical bug is found in the main branch (`master`). You can **switch to the bug fix branch** using `git checkout` and fix the bug. Once the bug is fixed, you can switch back to the `search-feature` branch and continue working without losing your progress.

Important Commands:

- `git checkout <branch_name>`: Switches to an existing branch, like `git checkout bugfix` to fix the bug.
- `git checkout -b <new_branch>`: Creates a new branch and switches to it immediately. For example, `git checkout -b update-design` will create

and move to a new branch where you can work on a design update.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git checkout -b project_bugs
Switched to a new branch 'project_bugs'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_bugs)
$ git branch -a
master
* project_bugs
project_changes
```

3. Combining Work: Merge

After working on a separate branch for a while (e.g., developing a new feature or fixing a bug), you will want to **combine your work** with the main project. This is where **git merge** comes in—it merges the changes from one branch into another, typically merging changes into the **master** branch.

- You've finished working on the `search-feature` branch, and it's time to **merge it into the master branch**. By using `git merge search-feature` while on the `master` branch, you combine all the changes from the `search-feature` branch into the `master`. Now, the `master` branch contains both the original stable version and the new feature you've developed.

Important Command:

- `git merge <branch_name>`: Merges the changes from the specified branch into the current branch. For example, `git merge search-feature` combines the search feature with the master branch.

```

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ nano b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git add .
warning: LF will be replaced by CRLF in b.txt.
The file will have its original line endings in your working directory

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git commit -m "New file added in new branch project_changes"
[project_changes e525410] New file added in new branch project_changes
 1 file changed, 1 insertion(+)
 create mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt b.txt

```

```

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git checkout master
Switched to branch 'master'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ ls
a.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git merge project_changes
Updating 4e87650..e525410
Fast-forward
 b.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ ls
a.txt b.txt

```

Activ

4. Viewing History: Log

Git keeps a **detailed history** of all changes (called **commits**) made to the project. The **git log** command shows you this history, so you can see who made changes, when they made them, and what the changes were. This is useful for tracking the progress of the project and

identifying when bugs were introduced.

- Suppose you want to know when a specific feature was added or who worked on a particular part of the code. By using the `git log` command, you can see a detailed history of all the changes. Each entry in the log will show:
 - **Commit Hash:** A unique identifier for each change.
 - **Author:** The person who made the change.
 - **Date:** When the change was made.
 - **Message:** A short description of the change, like “added search functionality.”

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git log
commit e525410c637e693b0b512bbb2fe67ca8c3763b14 (HEAD -> master)
Author: Madhurika, Budaraju <budarajumadhurika@gmail.com>
Date:   Wed Oct 16 00:16:39 2024 +0530

    New file added in new branch project_changes

commit 4e87650a0f49e0b6f82b53e693099f5987eeb972 (project_bugs)
Author: Madhurika, Budaraju <budarajumadhurika@gmail.com>
Date:   Tue Oct 15 14:37:32 2024 +0530

    Created new file
```

- **Simplified View:**

If you don't want to see too much detail, you can use the `git log --oneline` command. This shows a summary of each commit, making it easier to quickly scan through the project's history.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git log --oneline
e525410 (HEAD -> master) New file added in new branch project_changes
4e87650 (project_bugs) Created new file
```

Important Commands:

- `git log`: Shows the complete history of commits in a branch.
- `git log --oneline`: Shows a simpler view with one line per commit, like `f3b9c2b Added search feature.`

5. Undoing Changes: Revert

If a mistake was made in a previous commit, Git provides a way to **undo those changes** using the `git revert` command. This doesn't delete the commit but creates a new commit that cancels out the earlier changes. This way, the history of the project is preserved.

- Imagine you've added a new feature but later discover that it introduced a bug. Instead of deleting that commit, you can use `git revert` to undo the changes made in that commit. For example, if the commit hash for the problematic commit is `f3b9c2b`, you would use `git revert f3b9c2b`. Git will then create a new commit that undoes the changes introduced by that commit without deleting it from the project history.

Important Command:

- `git revert <commit_hash>`: Reverts (undoes) the changes from a specific commit. For example, `git revert f3b9c2b` will undo the changes made in the commit with the ID `f3b9c2b`.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git log --oneline
e525410 (HEAD -> project_changes, master) New file added in new branch project_changes
4e87650 (project_bugs) Created new file

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git revert e525410
[project_changes 7c2e526] Revert "New file added in new branch project_changes"
1 file changed, 1 deletion(-)
delete mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git status
On branch project_changes
nothing to commit, working tree clean

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ ls
a.txt
```

Flow of Using Git Commands (with Examples):

1. Create a Branch:

You create a branch to work on a new feature:

```
git branch new-feature
```

2. **Switch to the Branch:**

You switch to the new branch to start working on the feature:

```
git checkout new-feature
```

3. **Develop Your Feature and Commit**

Changes: You make changes, then commit them:

```
git commit -m "Developed search functionality"
```

4. **Switch to Another Branch to Fix a Bug:**

Suddenly, you need to fix a bug, so you switch to the bug fix branch:

```
git checkout bugfix
```

5. **Merge the Bug Fix Into the Main Branch:**

Once the bug is fixed, you merge it into the master branch:

```
git checkout master
```

```
git merge bugfix
```

6. **Check History Using Log:**

You want to review the commits to make sure everything is correct:

```
git log
```

7. **Revert a Mistake:**

If you discover a mistake in a previous commit, you can revert it:

```
git revert f3b9c2b
```

GIT COMMANDS: WORKING WITH REMOTE REPOSITORIES - REMOTE, CLONE, PULL, PUSH, FORK

1. Git Remote

Git remote allows you to manage connections to **remote repositories**, which are Git repositories hosted on the internet (like **GitHub** or **Bitbucket**). A remote repository is where you can save or retrieve your project from a server, and you use this command to add, remove, or view the addresses (URLs) of these repositories.

- To add a new remote repository, you use:

```
git remote add origin https://github.com/user/repo.git
```

This adds a new connection to a remote repository called `origin`, which is the common default name for remote repositories. After setting up the remote, you can start interacting with it, like pushing or pulling code.

- **Important Commands:**
 - `git remote -v`: Shows the current remotes and their URLs.
 - `git remote add <name> <url>`: Adds a new remote connection.
 - `git remote remove <name>`: Removes an existing remote.

The image shows two screenshots of the GitHub website. The top screenshot is the GitHub Home page, and the bottom screenshot is the 'Create a new repository' page.

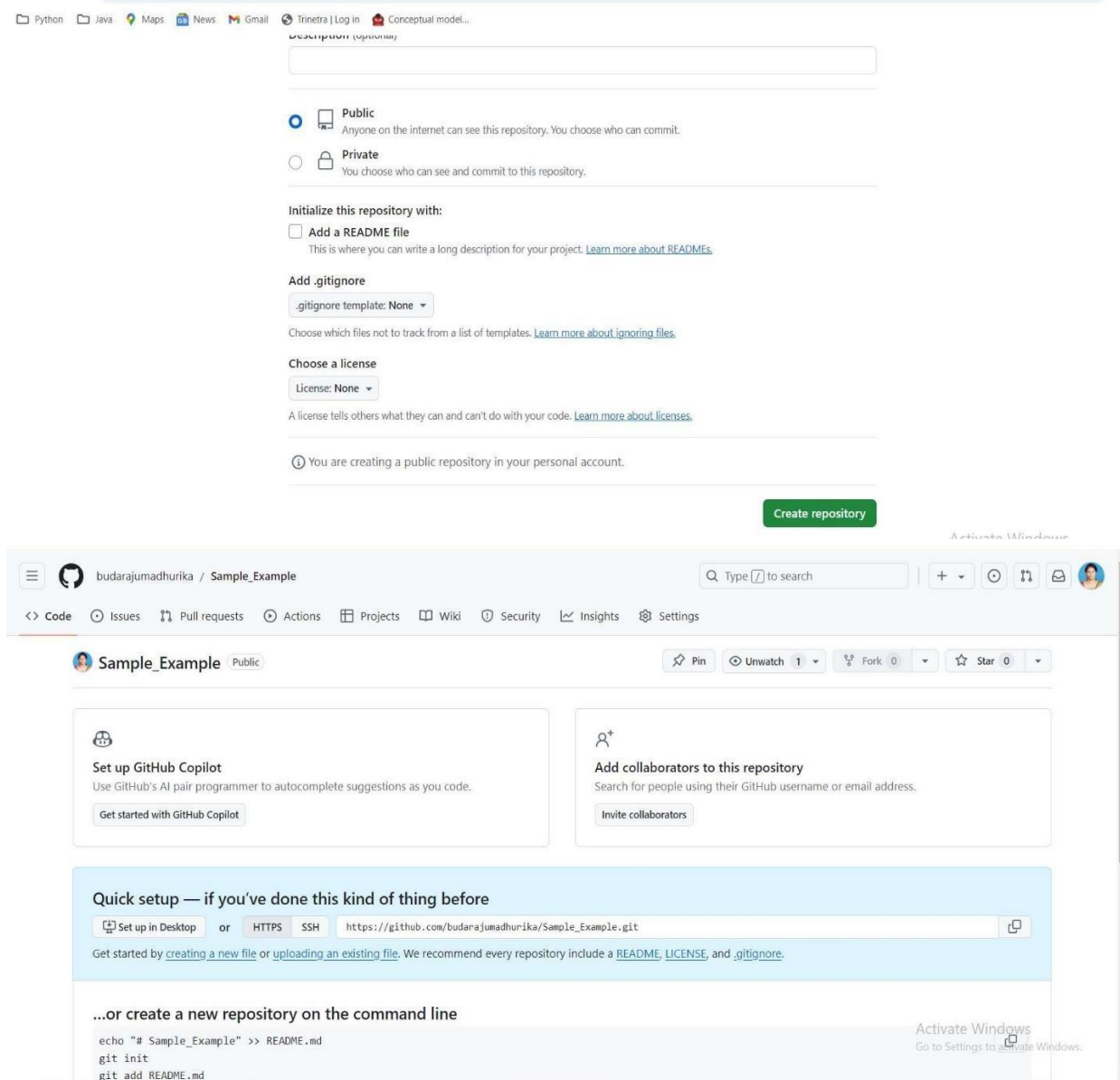
Top Screenshot: GitHub Home

- Header:** Includes navigation links (Python, Java, Maps, News, Gmail, Trinetra | Log in, Conceptual model...), a search bar, and a user profile icon.
- Left Sidebar:** Titled 'Top repositories', it lists several repositories by user 'budarajumadhurika', including 'samplepollscm', 'samplescriptjava', 'SrikantSample', 'JavaMaven', '21-12', '11-12-java', and 'WebDeployment'. A 'New' button is visible.
- Main Content Area:** Titled 'Home', it features a 'Trending repositories' section. Two repositories are highlighted:
 - openai/swarm:** Described as an 'Educational framework exploring ergonomic, lightweight multi-agent orchestration. Managed by OpenAI Solution team.' It has 12k stars and is written in Python.
 - SWivid/F5-TTS:** Described as 'Official code for "F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching"'. It has 3.4k stars and is written in Python.

Bottom Screenshot: Create a new repository

- Header:** Similar to the top screenshot, with a search bar and user profile icon.
- Main Content Area:**
 - Title:** 'Create a new repository'.
 - Text:** 'A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)'
 - Required fields:** 'Owner' (selected as 'budarajumadhurika') and 'Repository name' (input as 'Sample_Example'). A green checkmark indicates 'Sample_Example is available'.
 - Description:** An optional text field.
 - Visibility:** Two radio buttons: 'Public' (selected) and 'Private'.
 - Initialize this repository with:** A checkbox for 'Add a README file'.

Right Sidebar: Contains an 'Activate Windows' watermark with the text 'Go to Settings to activate Windows.'



The screenshot shows the GitHub 'Create new repository' form and the subsequent repository page for 'Sample_Example'.

Create new repository form:

- Repository name: (empty)
- Visibility: ☒ **Public** (Anyone on the internet can see this repository. You choose who can commit.)
- ☐ **Private** (You choose who can see and commit to this repository.)
- Initialize this repository with:
 - ☐ **Add a README file** (This is where you can write a long description for your project. [Learn more about READMEs](#).)
- Add .gitignore
 - .gitignore template: **None**
 - Choose which files not to track from a list of templates. [Learn more about ignoring files](#).
- Choose a license
 - License: **None**
 - A license tells others what they can and can't do with your code. [Learn more about licenses](#).
- ☐ You are creating a public repository in your personal account.
- Create repository** button

Repository page (Sample_Example):

- Navigation: Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings
- Repository name: **Sample_Example** (Public)
- Actions: Pin, Unwatch (1), Fork (0), Star (0)
- Quick setup — if you've done this kind of thing before:
 - Set up in Desktop or HTTPS SSH: `https://github.com/budarajumadhurika/Sample_Example.git`
 - Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).
- ...or create a new repository on the command line:

```
echo "# Sample_Example" >> README.md
git init
git add README.md
```

The top part of the image shows the GitHub repository setup page. It includes options to 'Set up GitHub Copilot' and 'Add collaborators to this repository'. Below these is a 'Quick setup' section with a text input field containing the repository URL: `https://github.com/budarajumadhurika/Sample_Example.git`. It also provides links for 'Set up in Desktop', 'HTTPS', and 'SSH'. A section titled '...or create a new repository on the command line' shows a series of terminal commands: `git init`, `git add README.md`, `git commit -m "first commit"`, `git branch -M main`, `git remote add origin https://github.com/budarajumadhurika/Sample_Example.git`, and `git push -u origin main`. Another section titled '...or push an existing repository from the command line' is partially visible.

The bottom part of the image shows a Windows File Explorer window. The address bar indicates the path: `This PC > Desktop > AY-23-24-Sem-2 > RKR21-SE > Tesseract > Sample_Example`. The main pane shows a file named `a.txt` with a size of 1 KB, modified on 10/16/2024 at 12:42 AM. A context menu is open over the file, showing options like 'View', 'Sort by', 'Group by', 'Refresh', 'Customize this folder...', 'Paste', 'Paste shortcut', 'Undo Rename', 'Git GUI Here', 'Git Bash Here', 'Open with Code', 'Give access to', 'New', and 'Properties'.

```

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch -M main

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote add origin https://github.com/budarajumadhurika/Sample_Example.git

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote -v
origin https://github.com/budarajumadhurika/Sample_Example.git (fetch)
origin https://github.com/budarajumadhurika/Sample_Example.git (push)

```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
seract/Sample_Example (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 248 bytes | 82.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/budarajumadhurika/Sample_Example.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

github.com/budarajumadhurika/Sample_Example

Python Reload this page News Gmail Trineira | Log in Conceptual model...

budarajumadhurika / Sample_Example

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Sample_Example Public

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot

Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH `https://github.com/budarajumadhurika/Sample_Example.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Sample_Example" >> README.md
git init
git add README.md
```

Activate Windows
Go to Settings to activate Windows.

budarajumadhurika / Sample_Example

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Sample_Example Public

main 1 Branch 0 Tags

Go to file t Add file <> Code

budarajumadhurika Added new file into master eaafdb9 · yesterday 1 Commit

a.txt Added new file into master yesterday

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

1 watching

0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#) to Settings to activate Windows.

2. Connecting Git to GitHub Using SSH Key

SSH (**Secure Shell**) keys are a way to connect your computer to GitHub without needing to enter a password each time.

It uses a **key pair**:

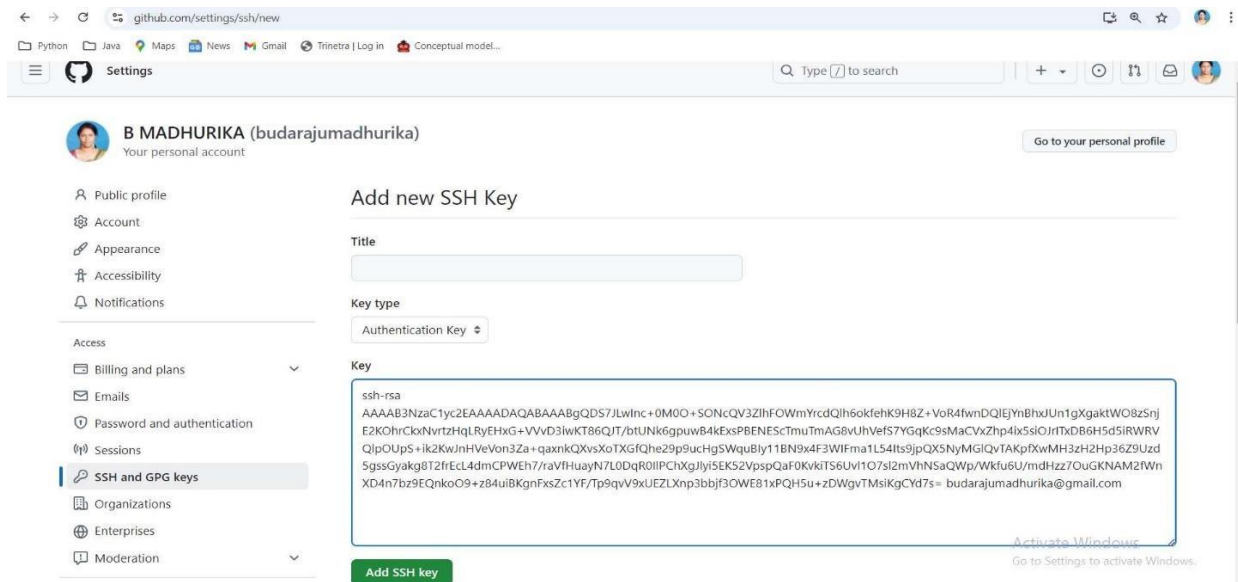
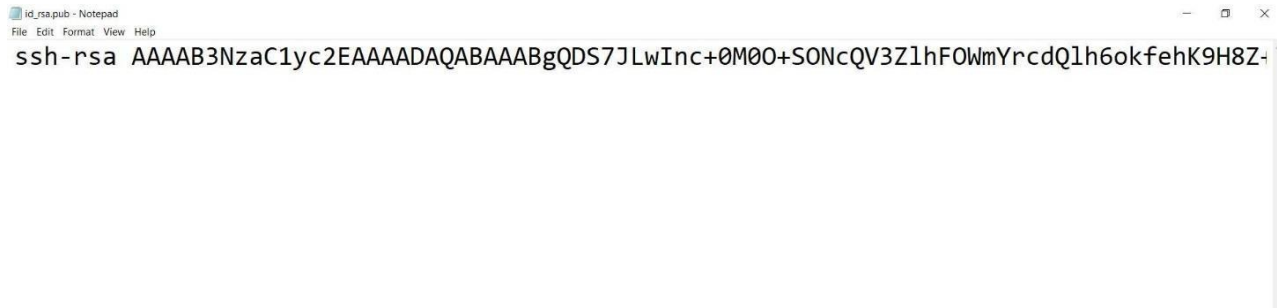
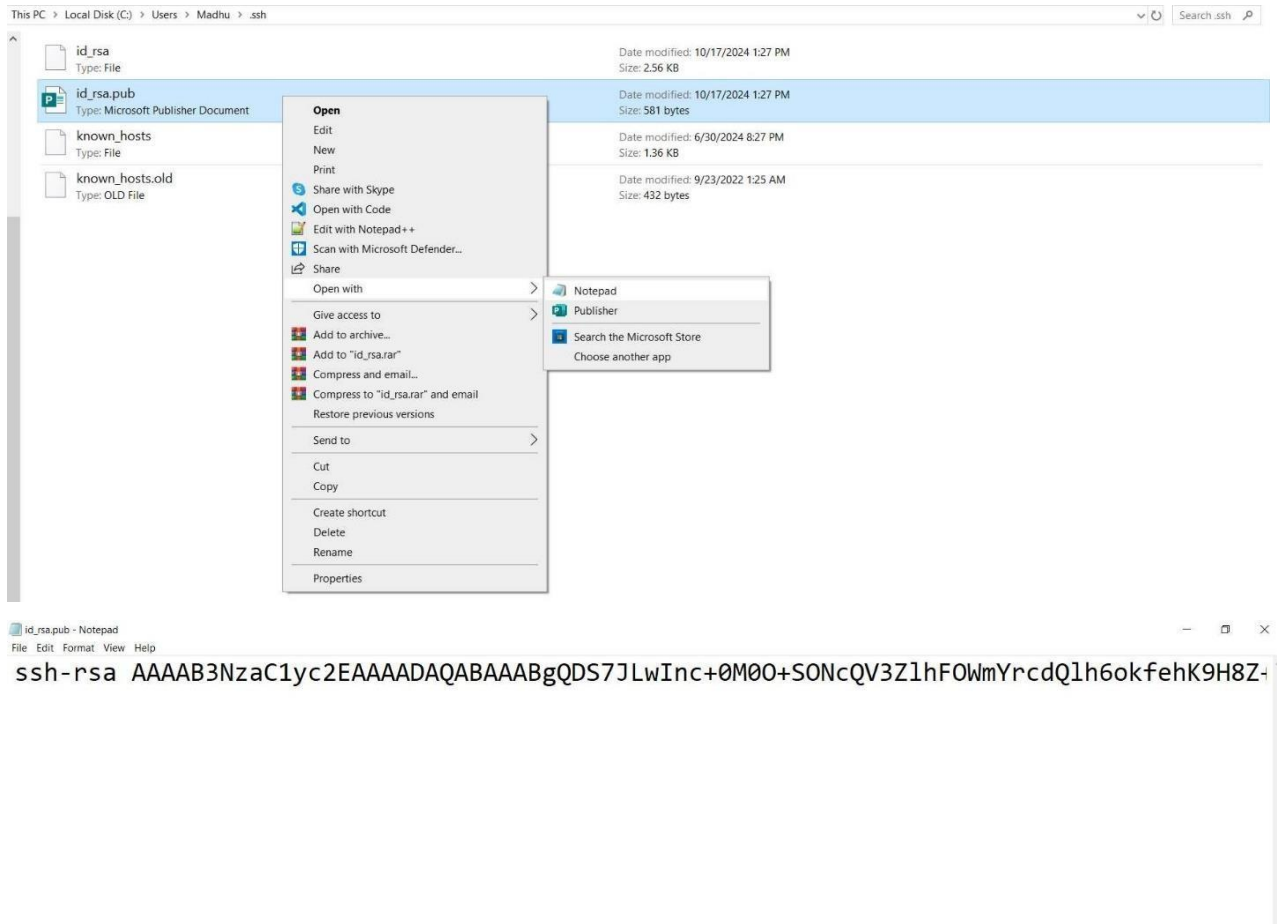
- A **private key** that stays on your computer.
- A **public key** that is shared with GitHub to verify your identity.
- **Steps for Generating SSH Key:**
 1. **Generate SSH Key:** Use the command `ssh-keygen -t rsa` to generate the key on your system.
 2. **Copy the Public Key:** The public key is stored in a file (usually `id_rsa.pub`), and you copy this key.
 3. **Add to GitHub:** In GitHub, go to **Settings > SSH and GPG keys > New SSH Key**. Paste the copied key into the text box and click “**Add SSH key.**”

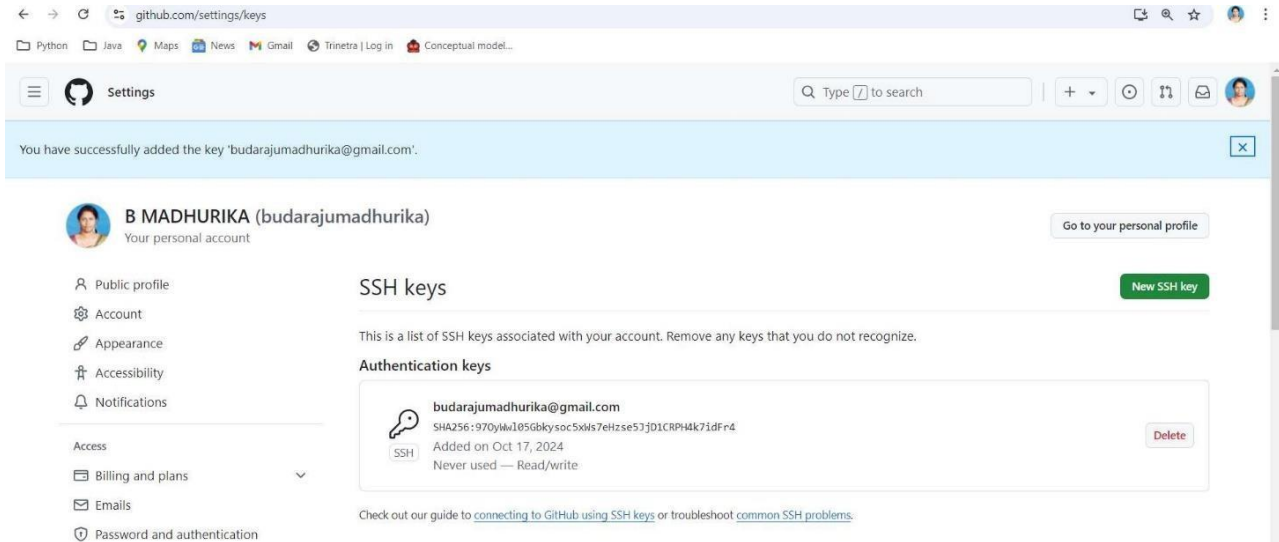
This allows you to securely interact with GitHub without typing your password every time.

```

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tes
seract/Example (master)
$ ssh-keygen -t rsa -C budarajumadhurika@gmail.com
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/Madhu/.ssh/id_rsa):
/c/Users/Madhu/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/Madhu/.ssh/id_rsa
Your public key has been saved in /c/Users/Madhu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:970ywwl05Gbkysoc5xws7eHZse5JjD1CRPH4k7idFr4 budarajumadhurik
a@gmail.com
The key's randomart image is:
+---[RSA 3072]---+
|                 =.                 |
|                B O                 |
|               . @ .                 |
|              o O + .                 |
|             S B * =                  |
|            o * B X +                 |
|           + . % @                   |
|          . . O *                    |
|         o+.oE                      |
+---[SHA256]-----+

```





3. Git Clone

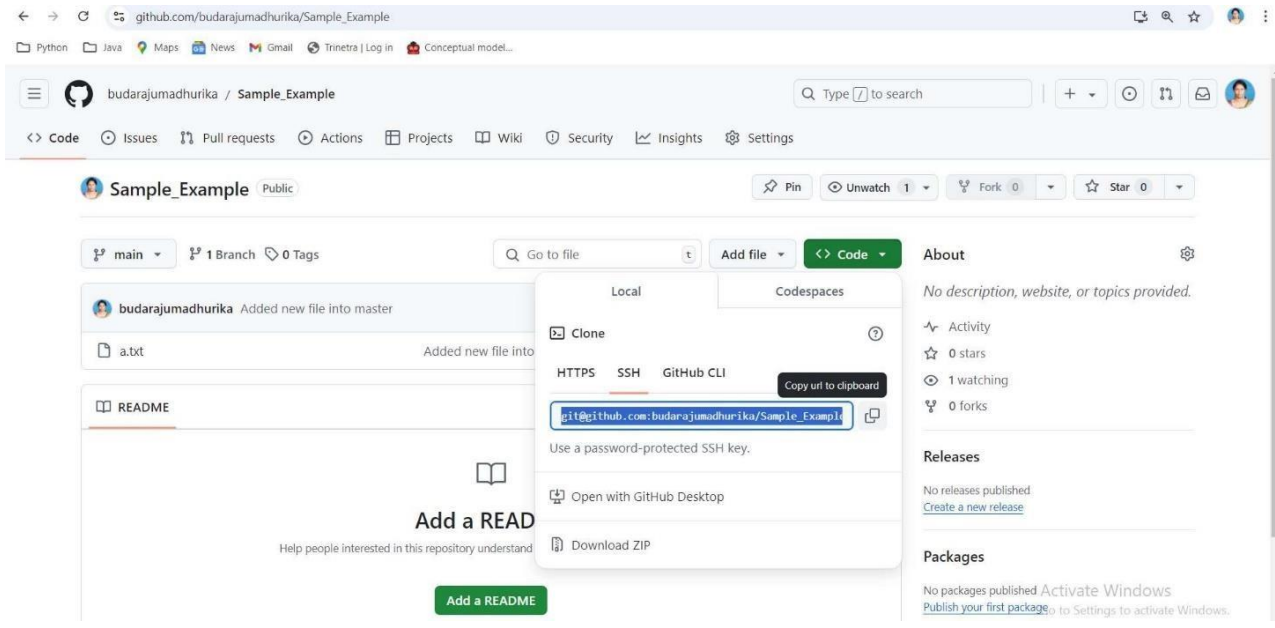
The **git clone** command is used to **download** a copy of a project from a remote repository to your computer. It creates a local version of the project that you can start working on right away.

- If you want to work on a project hosted on GitHub, you would use:

```
git clone https://github.com/user/repo.git
```

This creates a local copy of the project so you can begin editing the code on your computer.

- **Other Commands:**
 - **ls -ltr**: Lists the files in the current directory, showing the latest modified files after the cloning is done.

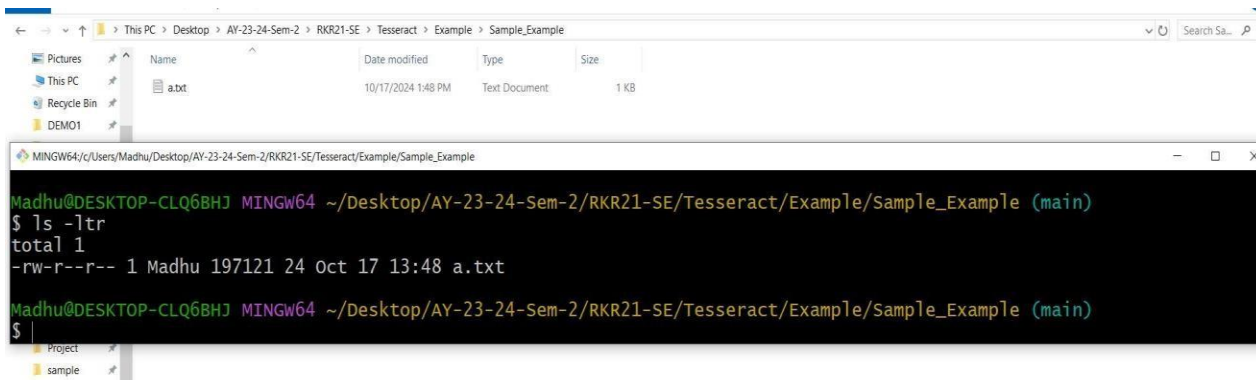


```
MINGW64:/c:/Users/Madhu/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example (master)
$ git clone git@github.com:budarajumadhurika/Sample_Example.git
Cloning into 'Sample_Example'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example (master)
$ ls -ltr
total 0
drwxr-xr-x 1 Madhu 197121 0 Oct 17 13:48 Sample_Example/

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example (master)
$ cd Sample_Example

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$
```



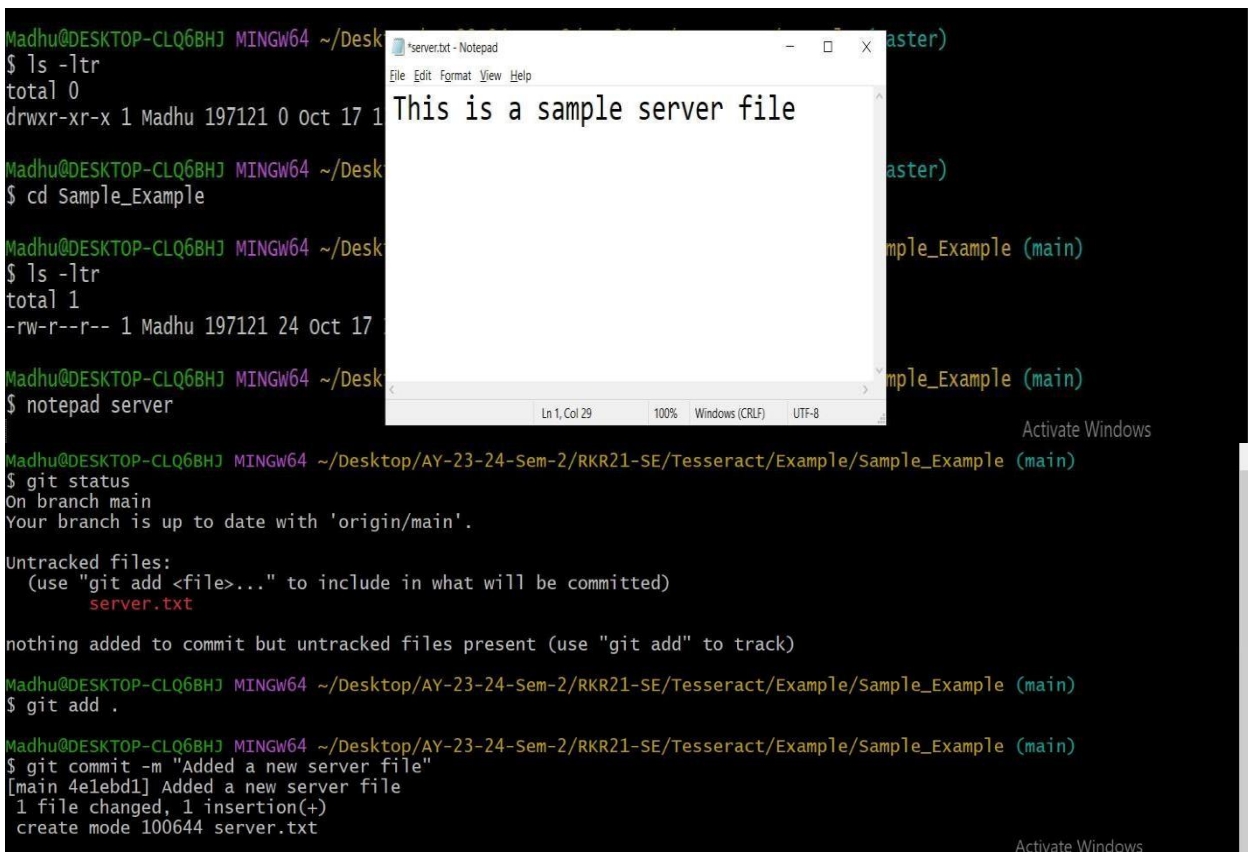
4. Git Push

After making changes locally, you use the **git push** command to send these changes from your local repository to the **remote repository** (for example, back to GitHub). This is how you **share your work** with others or keep your remote repository updated with your changes.

- After editing files on your computer and committing the changes, you would push them to the remote repository using:

```
git push origin master
```

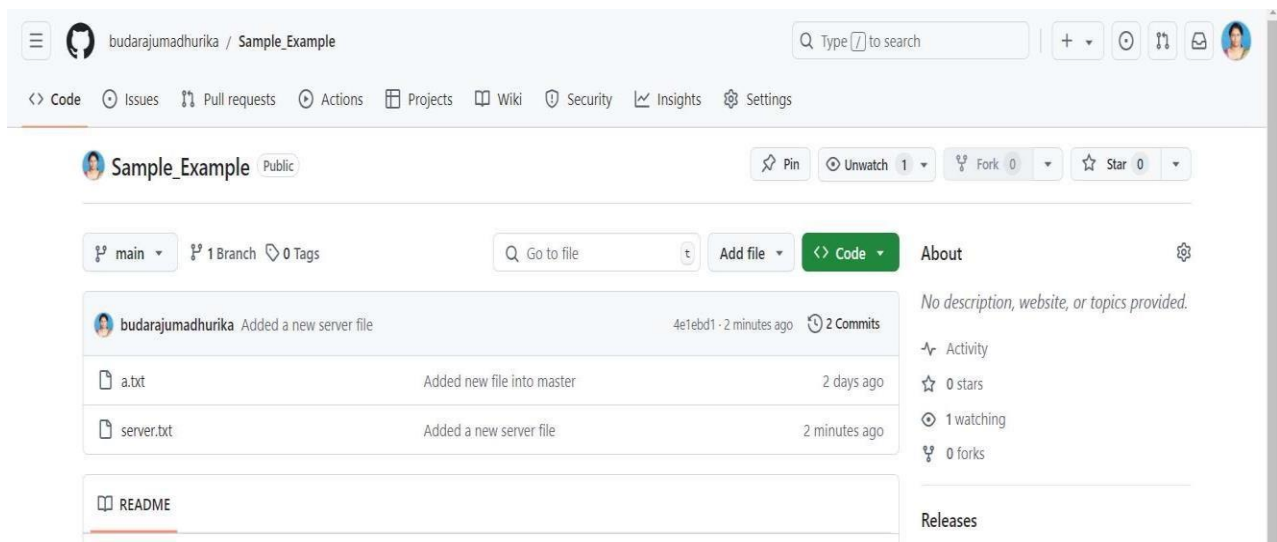
This pushes the changes from your local `master` branch to the `origin` remote (which is the GitHub repository).



```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git remote -v
origin  git@github.com:budarajumadhurika/Sample_Example.git (fetch)
origin  git@github.com:budarajumadhurika/Sample_Example.git (push)

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:budarajumadhurika/Sample_Example.git
   eaafdb9..4e1ebd1  main -> main

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Example/Sample_Example (main)
$
```



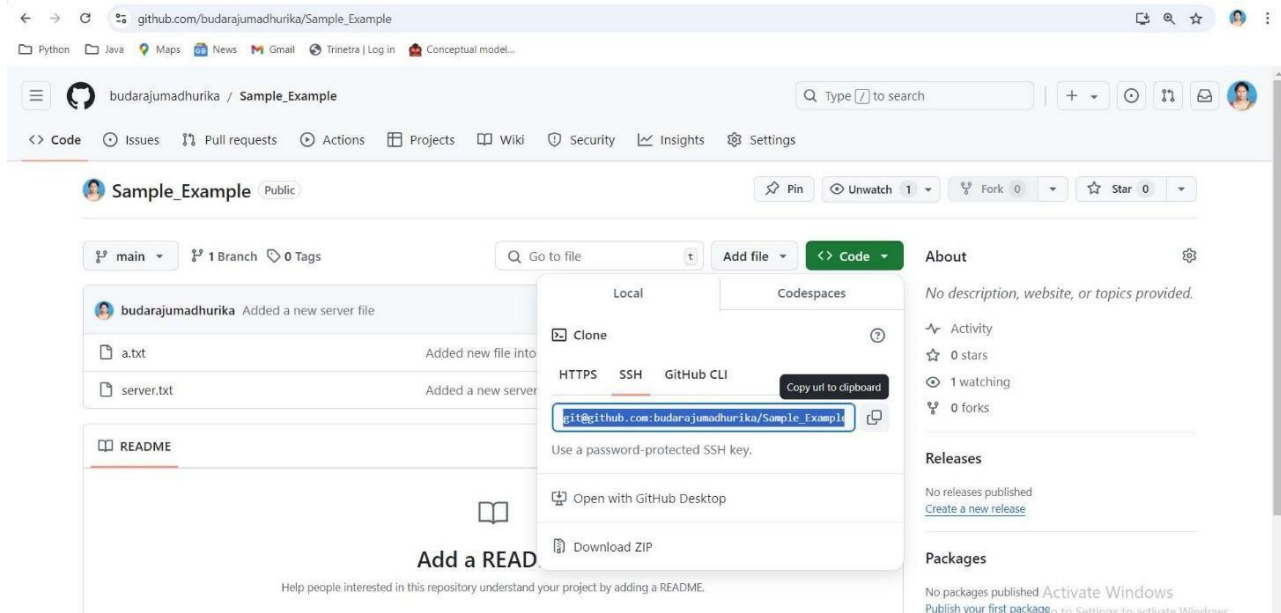
5. Git Pull

The **git pull** command fetches changes from the remote repository and merges them into your local repository. This ensures that you have the latest version of the project, including updates made by your teammates or collaborators.

- If you're working on a project with others and want to get the latest updates they made on GitHub, you would use:

```
git pull origin master
```

This will download any new commits from the `master` branch of the remote repository(`origin`) and merge them with your local code.



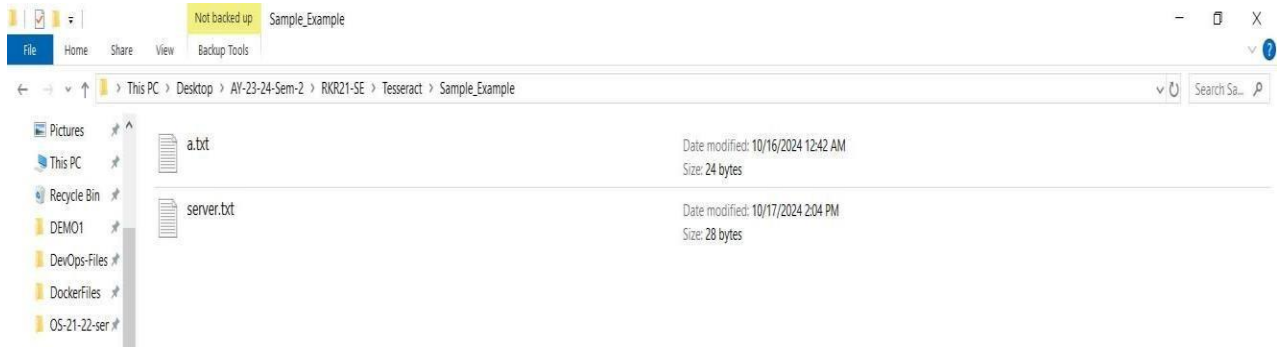
```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git remote -v
origin https://github.com/budarajumadhurika/Sample_Example.git (fetch)
origin https://github.com/budarajumadhurika/Sample_Example.git (push)

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ ls -ltr
total 1
-rw-r--r-- 1 Madhu 197121 24 Oct 16 00:42 a.txt
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ git pull origin main
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 294 bytes | 21.00 KiB/s, done.
From https://github.com/budarajumadhurika/Sample_Example
 * branch            main       -> FETCH_HEAD
    eaafdb9..4e1ebd1  main       -> origin/main
Updating eaafdb9..4e1ebd1
Fast-forward
 server.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 server.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (main)
$ ls -ltr
total 2
-rw-r--r-- 1 Madhu 197121 24 Oct 16 00:42 a.txt
-rw-r--r-- 1 Madhu 197121 28 Oct 17 14:04 server.txt
```

Activate Windows



6. Git Fork

A **fork** is a personal copy of someone else's repository, typically used in open-source projects. When you **fork** a repository, you get your own version of the project on GitHub that you can edit without affecting the original project. You can then propose changes to the original project by making a **pull request**.

- To fork a repository, go to the original repository on GitHub and click the **"Fork"** button. This makes a copy of the repository under your GitHub account where you can make your own changes.



github.com/orf/simple

Python Java Maps News Gmail Trinetra | Log in Conceptual model...

orf / simple

Type to search

Code Issues 4 Pull requests 2 Actions Projects Wiki Security Insights

This repository has been archived by the owner on Oct 26, 2018. It is now read-only.

simple Public archive

Watch 27 Fork 178 Star 505

Fork your own copy of orf/simple

master 4 Branches 0 Tags

Go to file

Code

About

Simple is a clone of Obvtse written in Python running on Flask.

Readme MIT license Activity 505 stars 27 watching 178 forks Report repository

orf Merge pull request #54 from adamchainz/patch-1 f64b392 · 6 years ago 278 Commits

simple	Change url protocol for more secure access	8 years ago
.gitignore	Bump version and fix nginx_config from not working	10 years ago
LICENSE	Add MIT license	10 years ago
MANIFEST.in	Add Manifest.in	10 years ago
README.md	Update README for maintenance status	6 years ago

orf / simple

Type to search

Code Issues 4 Pull requests 2 Actions Projects Wiki Security Insights

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

No available destinations to fork this repository.

[budarajumadhurika/simple](#)

github.com/budarajumadhurika/simple

Python Java Maps News Gmail Trinetra | Log in Conceptual model...

Code Pull requests 2 Actions Projects Wiki Security Insights Settings

simple Public

forked from orf/simple

Pin Watch 0 Fork 0 Star 0

master

Go to file

Code

About

Simple is a clone of Obvtse written in Python running on Flask.

Readme MIT license Activity 0 stars 0 watching 0 forks

Activate Windows
Go to Settings to activate Windows.

Releases

This branch is up to date with orf/simple:master. Contribute Sync fork

orf Merge pull request orf#54 from adamchai... f64b392 · 6 years ago 278 Commits

simple	Change url protocol for more secu...	8 years ago
.gitignore	Bump version and fix nginx_config...	10 years ago
LICENSE	Add MIT license	10 years ago
MANIFEST.in	Add Manifest.in	10 years ago

Detailed Flow with Examples

1. **Clone a Repository:** If you want to contribute to an open-source project or collaborate with others, you first clone the repository to your computer:

```
git clone https://github.com/user/repo.git
```

2. **Make Changes Locally:** Once you have the project on your local machine, you can make changes to the files, such as fixing bugs or adding new features. After editing, you commit the changes:

```
git commit -m "Added new feature"
```

3. **Push Changes to Remote:** After committing your changes, you push them back to the remote repository (GitHub):

```
git push origin master
```

4. **Pull Changes Made by Others:** If someone else makes changes to the project while you're working on it, you can pull those updates to ensure your version is up-to-date:

```
git pull origin master
```

5. **Use SSH Keys for Security:** Instead of typing your password every time you push or pull changes, you can set up an SSH key to make this process secure and automatic:

```
ssh-keygen -t rsa # Generate SSH key
```

6. **Forking a Repository:** If you want to make your own changes to someone else's project (such as in open-source), you fork the repository first, creating your own copy:
 - Visit the original project on GitHub.
 - Click the **Fork** button.
 - Make your changes in your personal copy, and then propose your updates back to the original repository by creating a **pull request**.

Git Patch: A patch in Git is a text file that represents changes between two sets of files, or commits. It's essentially the output of the `git diff` command, packaged in a format that can be applied to another set of files.

git format-patch -1 commit-hash-code

In `git format-patch -1 <commit-hash>`,

- `-1` means "create a patch from 1 commit"
- This creates a `.patch` file for a single commit.
- Example:

git format-patch -1 abc1234

- This creates a file like `0001-Your-commit-message.patch`

To create patches for the **last 3 commits** the command is: `git format-patch -3`

Once the remote owner gets the patch file 0001-commit-from-cloned-person.patch next the owner need to apply patch file in his git bash using git command below:

Syntax:

git apply my-changes.patch

Or, if it's a patch made by `git format-patch`, use:

git apply 0001-Your-commit-message.patch

Git Merge Conflict: A merge conflict occurs when Git cannot automatically combine changes from different branches due to conflicting edits, requiring manual resolution by the developer.

Merge conflicts usually arise in the following scenarios:

1. **Same line changed in two branches:** If two people edit the same line in a file in different branches, Git can't choose which version is correct.

Example:

- **Branch A:** Welcome to our website!
- **Branch B:** Hello from our website!

Git gets confused because both changed the same line.

2. **One branch deletes a file, another edits it:** If one branch deletes a file and another changes it, [Git](#) doesn't know whether to keep the file or remove it.

3. **File renamed in one branch, edited in another:** If a file is renamed in one branch and edited in another, Git doesn't know how to join those changes.

Imagine there is a file called `notes.txt`, and it has this content:

```
Hello, this is the project note.
```

Person A (on main branch) changes the file like this

```
Hello, this is the **updated project note** by A.
```

Then commits the change.

Person B (on feature branch) changes the same line

Hello, this is the ****feature note**** by B.

Then commits the change.

Now Person A tries to merge Person B's branch (feature) into main

```
git merge feature
```

But Git gets confused! Both branches changed the same line differently. Git doesn't know whose change to keep.

Git shows a merge conflict in notes.txt like this

```
<<<<<<< HEAD
Hello, this is the **updated project note** by A.
=====
Hello, this is the **feature note** by B.
>>>>>>> feature
```

- ===== separates the conflicting changes.
- Content above it belongs to the current branch (HEAD / master).
- Content below it belongs to the merging branch (feature).

Types of Merge Conflicts:

The below are different types of merge conflicts in git:

1. **Content Conflicts (Edit–Edit):** This happens when two people change the same line in a file in different branches. Git doesn't know which version to keep.
2. **Delete–Modify Conflicts:** One branch deletes a file, and the other branch makes changes to it. Git gets confused should the file be kept or deleted?
3. **Rename–Modify/Rename–Delete Conflicts:** One branch renames a file, while the other branch edits or deletes it. Git can't track the file properly because the name is different.
4. **Directory–File Conflicts (Structural Conflicts):** One branch turns a file into a folder, while the other branch still uses it as a file. Git doesn't know if it's a file or folder.
5. **Submodule Conflicts:** If both branches point to different versions of a submodule, Git doesn't know which one to use.
6. **Binary File Conflicts:** If both branches make changes to a binary file (like an image or PDF), Git can't compare them line-by-line. So it asks you to choose which version to keep.
7. **File Mode Conflicts:** One branch changes a file's permissions (like making it executable), and the other branch doesn't. Git sees this as a conflict.
8. **Index Conflicts:** This happens inside Git's staging area when things are changed in a way Git doesn't know how to handle. Usually happens in more advanced Git operations like git rebase.